

Product Evaluation

Overview

- A release has gone to the client (?)
- Your code is clean. Modules integrate. All the tests pass. It meets the spec. It's doing clever stuff in an efficient and optimised way.
- Is that enough?
- Tests are about problems you can foresee. The world is too complex and messy to predict.
- No matter how clean and sophisticated the code is, if it creates friction or irritation for users the software is bad.

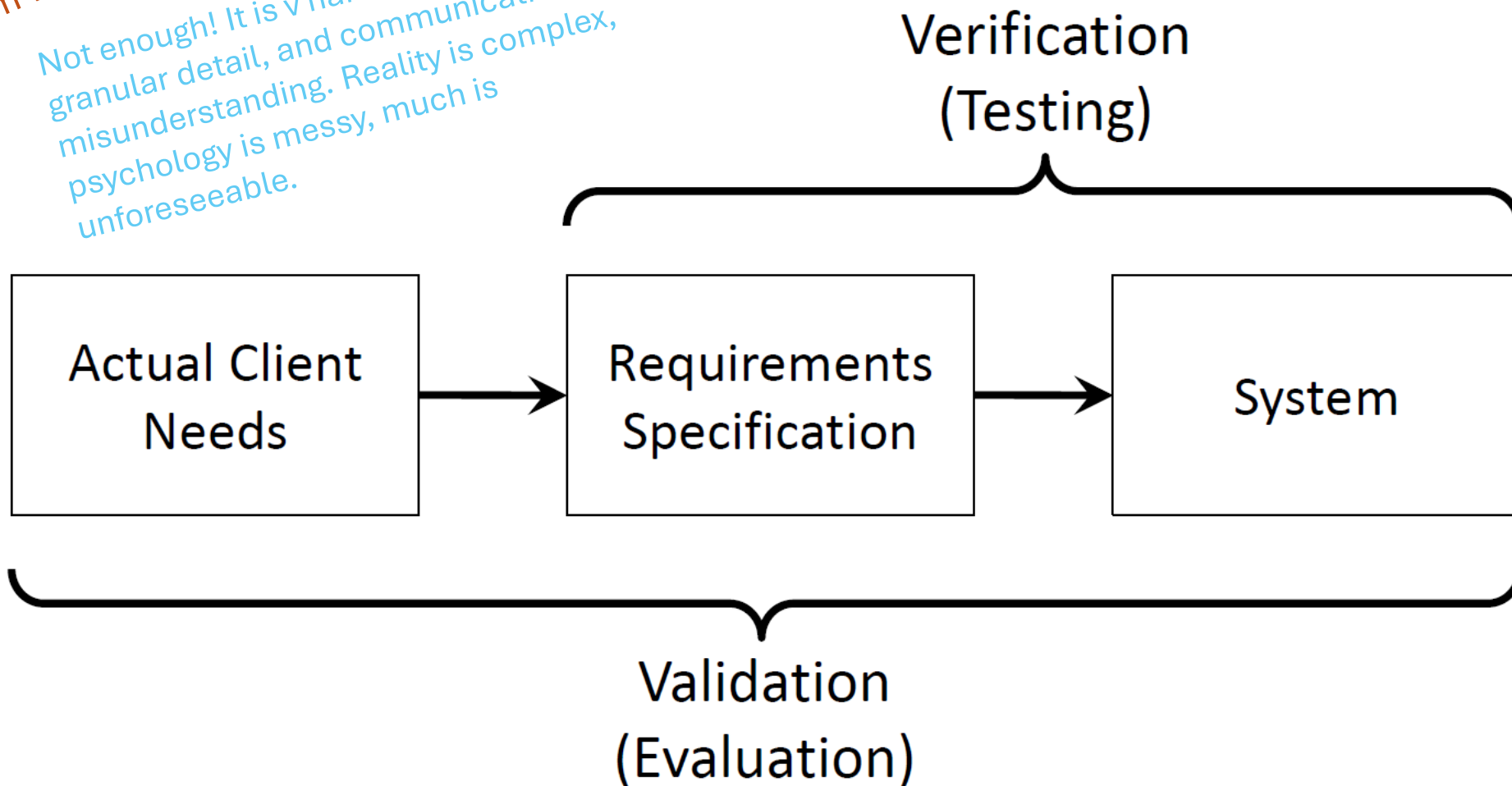
Not testing. Evaluation:

What happens when the software (*a clean, discrete, deterministic, thing*) interfaces with users and their needs (*which are messy, context dependent, nonlinear*)?

How does it fit?

“We asked them what they wanted!”

Not enough! It is v hard to articulate needs in granular detail, and communication entails misunderstanding. Reality is complex, psychology is messy, much is unforeseeable.



Human Computer Interaction (design, psychology, sociology, ergonomics, ...)

At minimum, we want our product to be useful

$$\text{Usefulness} = \text{Utility} \times \text{Usability}$$

What does it do?

Does it solve a real need?

Does it do something
people want to do?

Can people* learn the
interface

Can they operate it
efficiently*

* According to the Norman Nielsen Group
(Arguably the world's most successful
and influential design consultancy)

<https://www.nngroup.com/videos/usefulness-utility-usability/>

*Note: multiplication! if either term
is low, usefulness is low*

At minimum, we want our product to be useful

$$\text{Usefulness} = \text{Utility} \times \text{Usability}$$

What does it do?

Does it solve a real need?

Does it do something
people want to do?

Can people* learn the
interface

** The group of people who
will actually use it. Not
computer scientists!*

Can they operate it
efficiently*

**In the real context of use*

- Not what people *think* they need (*specs aren't enough*)
- Most people can't articulate need in detail (otherwise they'd be successful inventors)
- Requires trial in real contexts ("in the wild!"). How does functionality combine with tools, constraints and needs.

- Many dimensions
- Incredibly complex construct ...

Manager: "We need to parse this complex data format"

On the ground: "that format causes other problems. We want to move away from it"

What is required for usability?

Efficient?	Yes!	Safe	Yes!
Reliable?	Yes!	Secure?	Yes!
Satisfying?	UX (User Experience)	Configurable?	Yes!
Learnable?	Yes!	Fun?	UX (User Experience)
Maintainable?	Yes!	Available?	Yes!

Different aspects of this will be more or less important for different technologies

How can we understand the usability of our tech?

- How do we identify problems?
- How do we pinpoint them to different aspects of usability?
- Can we quantify usability?

1. Heuristic Evaluation

- Find a checklist of required characteristics: does it do this?
- Check this with multiple reviewers with relevant expertise
- Include external reviewers to avoid dev-team bias

A nice quick starting point BUT

- Relatively shallow – focus on static qualities
- Cant deal with complex, issues that emerge in real contexts and interactions.
- You still didn't actually talk to the users!

1

Visibility of System Status

The design should always keep users informed about what is going on, through appropriate feedback within a reasonable amount of time.

- Does the design clearly communicate its state?
- Is feedback presented quickly after user actions?

Issues

Recommendations

3

User Control and Freedom

Users often perform actions by mistake. They need a clearly marked "emergency exit" to leave the unwanted action without having to go through an extended process.

- Does the design allow users to go back a step in the process?
- Are exit links easily discoverable?
- Can users easily cancel an action?
- Is *Undo* and *Redo* supported?

Issues

Recommendations

<https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/>

Evaluation workbooks [here](#)

2. Task-based Evaluation (with screen recording)

- Choose a particular (relevant!) task your system should achieve
- Ask the user to try to achieve this and watch (carefully!) and take notes (in detail!)
- Must cover an “ecologically” wide range of activities (everyday but also exceptional “critical incidents”)

Like code-coverage in testing, task-covering is important here.

This requires a comprehensive list of use-cases, goals, and task-flows

What are we looking for?

How should we capture them?

- Quantitative aspects:

- Time
- Error rate
- Time-stamped action log
- “Human factors” questionnaires:
 - Cognitive load
 - Frustration

Requirements:

- you must already expect it
- It must be ... quantifiable
- It must be readily captured without too much intrusion

- Qualitative aspects:

- Kinds of frustration
- Accessibility issues
- Qualities of experience
- Improvisatory behaviours

Tools:

Free-text questionnaires

“talk aloud”

Interviews

Focus groups

Requires more skill and care to do well

More time-consuming

More informative

Captures unexpected issues

Quant Questionnaires

Seem easy. Actually quite hard to do well

- Allows you to quantify “human factors” and make statistical comparisons
- Designing quantitative questionnaires is a science
 - Do your questions bias users?
 - Handling measurement error (measurement theory!)
 - Validating new questionnaires requires sophisticated statistical techniques and large numbers of users.
- Psychologists have made and validated some scales for you to use (like SUS, NASA TLX)
 - But do they quantify the things you need to know? Are the scores meaningful for your product?

Likert (SUS)

Q1. I think I would like to use this system frequently.

☐ Strongly disagree ☐ Disagree ☐ Neither agree nor disagree ☐ Agree ☐ Strongly agree

Q2. I found the system unnecessarily complex.

☐ Strongly disagree ☐ Disagree ☐ Neither agree nor disagree ☐ Agree ☐ Strongly agree

Q3. I thought the system was easy to use.

☐ Strongly disagree ☐ Disagree ☐ Neither agree nor disagree ☐ Agree ☐ Strongly agree

Qual Questionnaires

Also seem easy. Also actually quite hard to do well

- Bias is still a risk – leading questions
- How are you going to analyse it?
- Short questions and responses can be superficial
- More in-depth questions and responses require careful and rigorous analysis.

Please tell us three things we can do to improve our service:

In just one phrase, how would you describe Qualtrics culture?

After interacting with Qualtrics, what suggestions can you give us to help create an engaging culture?

Structured Interviews

... harder than they look

- Similar to qualitative questionnaires but can be more responsive
- More in depth responses from users
- Requires a script of (semi) structured questions to work well
- Open-ended interviews are possible but require considerable skill, both in interview situation and during the analysis.
- Decide ahead of time how to analyse.

Focus Groups

... you get the point by now

- Like interviews but in a small group, and you guide, or facilitate discussion, rather than ask questions from a script
- Make sure everyone gets to speak
- The larger group encourages some people to speak more freely and bounce ideas off each other
- But individual perspectives can be lost
- Requires a confident and adaptive facilitator

Speak Aloud (with screen recording)

- Get the user to speak aloud as they perform a task, or use the technology freely (good to have both)
 - What they are thinking
 - What is hard to understand
 - Problems they encounter
 - Things that work well
- Advantages:
 - Doesn't rely on memory
 - Easy to connect problems to specific aspects of the software
 - Can make it easier to analyse
 - Uncovers issues we can't anticipate
- Disadvantages
 - Can prompt performance anxiety for users
 - Unreflective answers
 - Talking aloud can make behaviour unrealistic

What does a bad question look like?

Vague questions

Bad: “what would you give our product out of 10?”

Better: Literally any question which you have thought about for a moment or two.

Leading questions

Bad: “Which aspects of our phenomenal product did you love most”

Better: “Can you talk about three aspects of the product which were positive”

Loaded questions

Bad: “How did our technology help your customer service staff appear less incompetent?”

Better: “How did our technology improve customer service interactions?”

What does a bad question look like?

Leading questions

Bad: “Which aspects of our phenomenal product did you love most”

Better: “Can you talk about three aspects of the product which were positive”

Loaded questions

Bad: “How did our technology help your customer service staff appear less incompetent?”

Better: “How did our technology improve customer service interactions?”

What does a bad question look like?

Double barrelled questions:

Bad: “Did the new login page make processing faster and more secure?”

Better: “Did the new login page make processing faster?”

.... “Did the new login page make processing more secure?”

Absolute

Bad: “Do staff [*always/ever*] make use of the calculation tools currently provided?”

Better: “Do staff make use of the calculation tools currently provided?”

Irrelevant

Bad: “Do you prefer SHA-2 encryption to the previous MD5 implementation?”

Better: Literally anything relevant to the immediate interests and goals of the user. Avoid technical terms where not required. Focus on their goals and tasks.

What does a bad question look like?

Truisims:

Bad: “Is electronic messaging with our system faster than postal delivery?”

Better: “Is electronic messaging with our system adequately fast?”

Indeterminable

Bad: “Are the sales pages navigated in the most efficient order?”
“is the data storage mechanism secure”

The user isn’t the right person to ask. Ask about the impact on usage.

Jargon / technical

Bad: “Are multi-table joins fast in the JPA CRUD repository?”

Better: Literally anything relevant to the immediate interests and goals of the user. Avoid technical terms where not required. Focus on their goals and tasks.

Analysing Qual Data: basics

- Come to the interview session with a list of things to focus on
- One person interviews, another observes and takes notes
- Take short hand-written notes to guide later analysis:
 - Relevant events with rough timings, issues to focus on in analysis, etc.
- Also capture audio/video/screen
- First analysis – counting clicks, errors, noting timings
- Deeper analysis (on focused subset of data) – transcribe and pick out “themes”

Analysing Qual Data

“Content Analysis”-lite :

1. Transcribe and then read all the text
2. Identify statements relevant to an area of interest.
 1. Summarise each statement as a short sentence capturing the key point (a “code”, e.g. “can’t find icon”, “output is in incompatible format”.)
 2. Record in a spreadsheet next to the original statement
3. Identify “codes” which belong together. Group as a “theme” which captures a high level problem or positive issue (E.g. “navigation difficulties”, “interfacing with other systems”)
4. For each “theme”, write a high level narrative account summarising and interpreting the content grouped in it. Include some illustrative extracts where relevant.